

OBJEKTORIENTERET ANALYSE, DESIGN OG IMPLEMENTATION

3: Class Relations and Diagrams

Jan H. Mikkelsen
Cyber og Computerteknologi
2. semester – KBH – F26

AGENDA

- Class interactions and relations
 - Inheritance
 - Associations
 - Aggregations
 - Composition
- How to complete class diagrams
- Example
- The important stuff
- Assignments

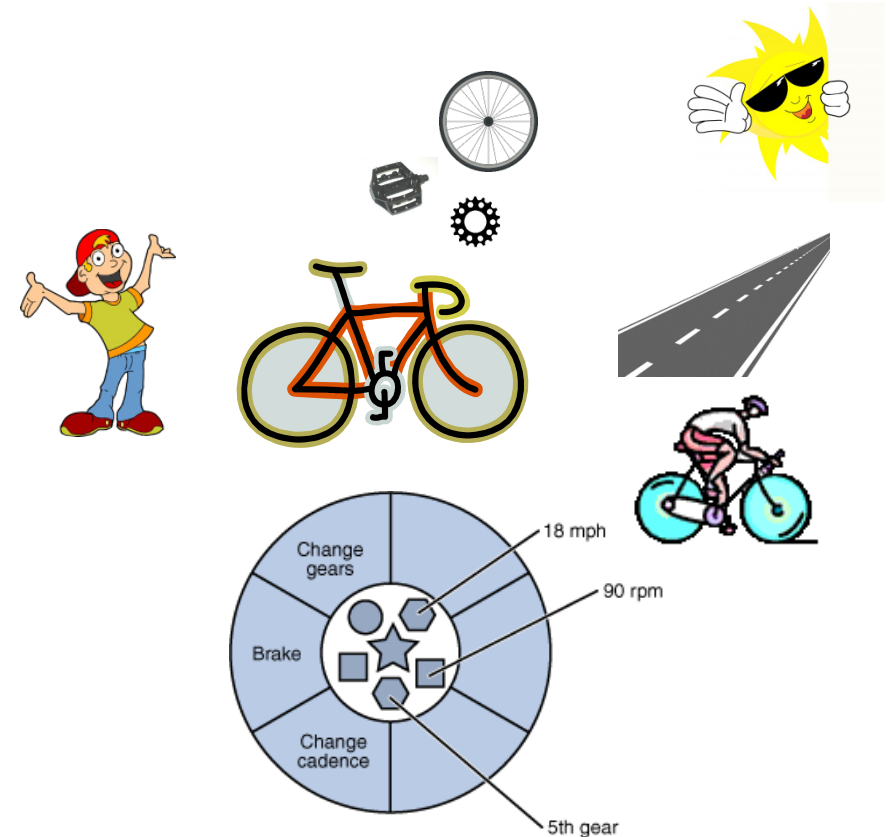


- To be able to design a larger system composed by classes
- To be able to utilize important aspects as inheritance in design
- To be able to organize and structure class relations that reflect code effectively

CLASS INTERACTIONS AND RELATIONS

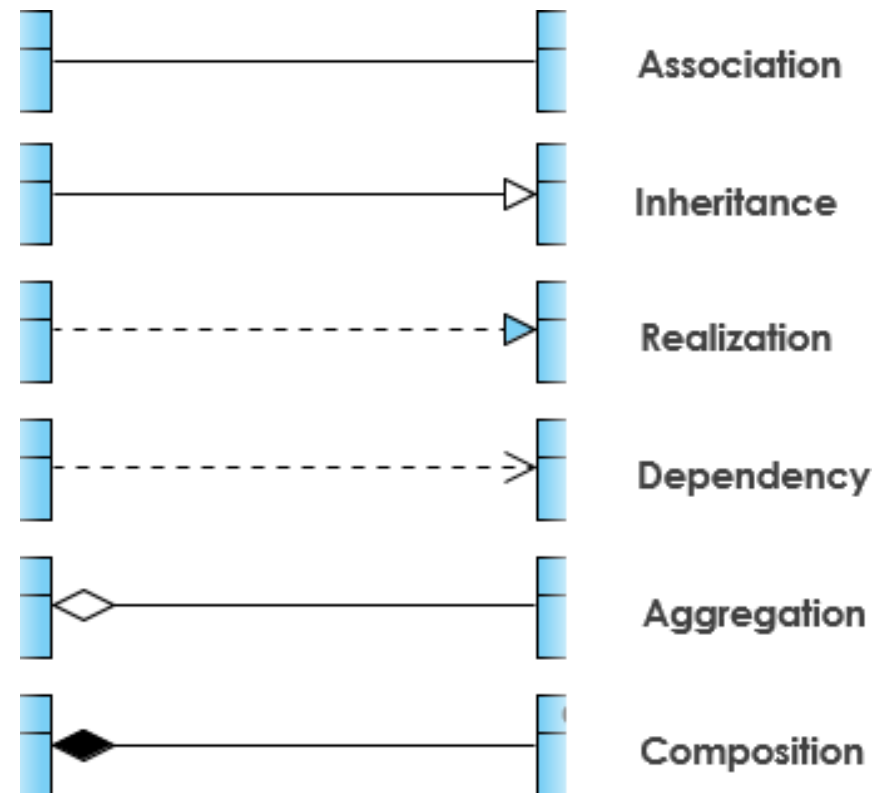
2

- We have a lot of objects, but somehow, they also relate to each other
 - As a person I may have a bicycle
 - I can do stuff with that bicycle
 - The bicycle is composed by smaller parts
 - The bicycle interacts with the road in different weather conditions. Sharp turns in icy conditions will feel quite a bit different than when done on nice warm and dry tarmac
 - There are other bicycles of other types
- We need to model this in some way



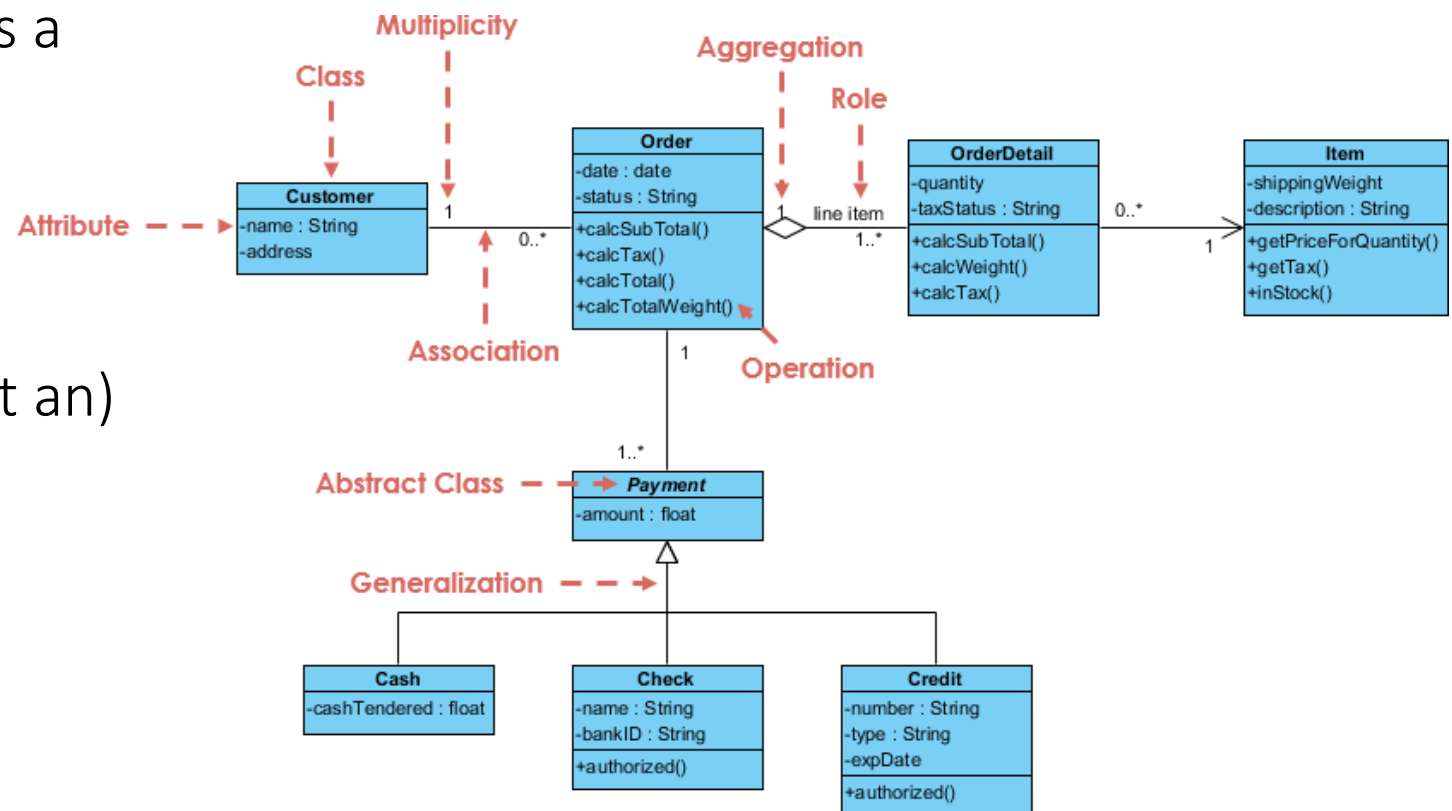
UML STANDARDS FOR CLASS DIAGRAMS

- UML specifies a set of different interactions and relation between classes
- We draw classes with these relations in *class diagrams*
- Useful to capture the static relations, however not dynamic (timely) relation
- This is taking things one step further compared to last lecture, where we mainly looked



CLASS DIAGRAMS – EXAMPLE ORDER SYSTEM

- An arrow indicates a direct association
- This requires communication and is therefore stronger than (just an) association



<https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/>

AGENDA

- Class interactions and relations
 - Inheritance
 - Associations
 - Aggregations
 - Composition
- How to complete class diagrams
- Example
- The important stuff
- Assignments

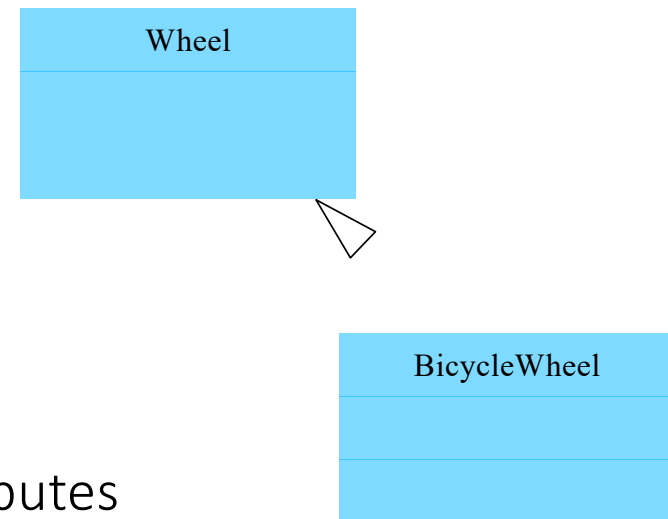
CLASS DIAGRAMS – INHERITANCE

6

- Inheritance (or generalization) is a key feature of OO

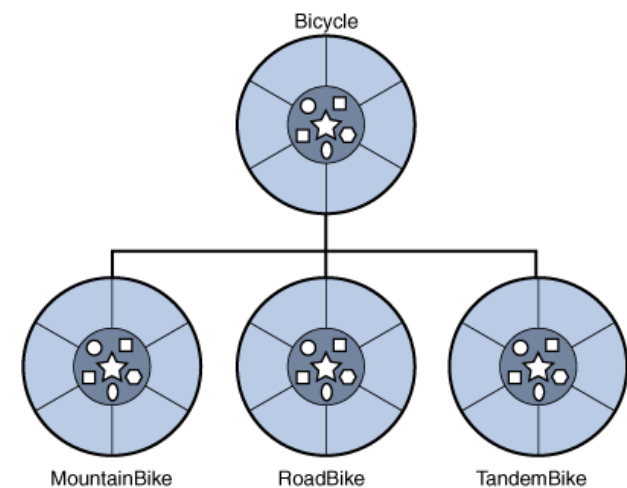
"A is a B" relationship

- A Bicycle wheel is also a Wheel, with all the characteristics of a wheel
 - BicycleWheel *is a* Wheel
- A CarWheel is also a Wheel, but it is NOT a BicycleWheel
- A Wheel needs to capture the common attributes and methods of both types of wheels
- BicycleWheel can invoke and provide methods in Wheel



INHERITANCE – WHAT IS THAT?

- Take, as an example, the case where any one person has different bikes? Do we need to repeat code to handle this?
To handle all the similar things?
- No, a strong part of OO is inheritance
 - A parent is generalizing its children, and children inherits a lot from its parents
- In practice, this means write code one place, and reuse it several places
 - This leads to better and often less code

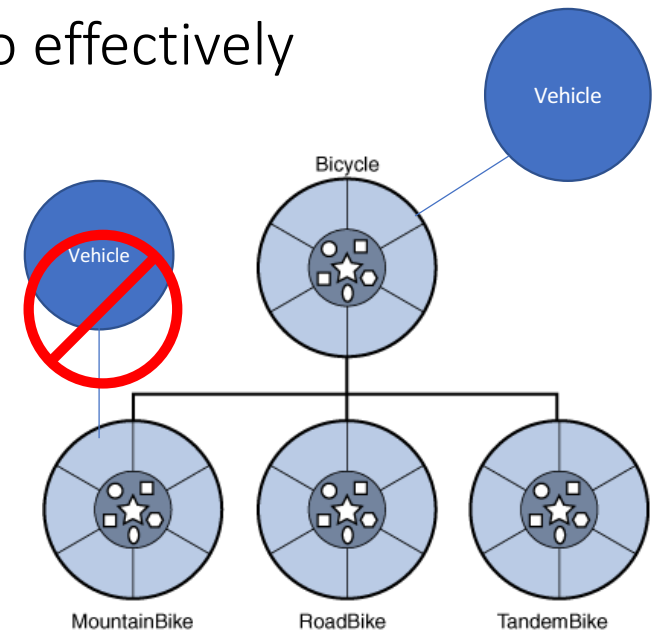


INHERITENCE – WHY?

- Inheritance is a strong concept that allows us to effectively reuse pieces of our code
- Inheritance relies on the keyword **extends**

```
public class MountainBike extends Bicycle
```

sub class super class

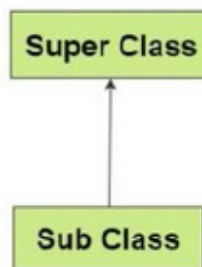


- In Java, we can only extend from one parent
 - If we need more, then we need a hierarchy, e.g. Vehicle -> Bicycle

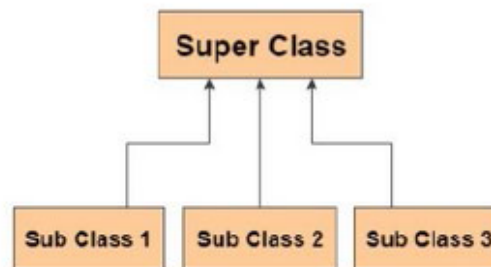
INHERITANCE

- Inheritance is such an important aspect of OO design that it often is seen as equivalent to OO
- It is also known as generalization, since the super class is preferably a generic version of a sub class
 - The generic version of a racer bike is a bicycle
 - The generic version of a bus is a vehicle
 - The generic version of a train is a means of transport, which is also a vehicle

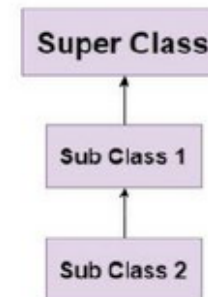
Single Inheritance



Hierarchical Inheritance

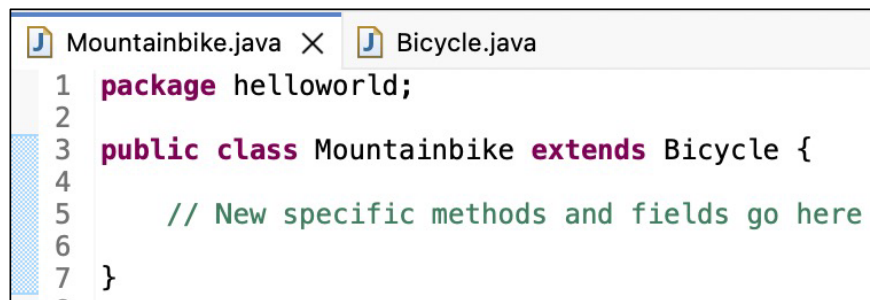


MultiLevel Inheritance

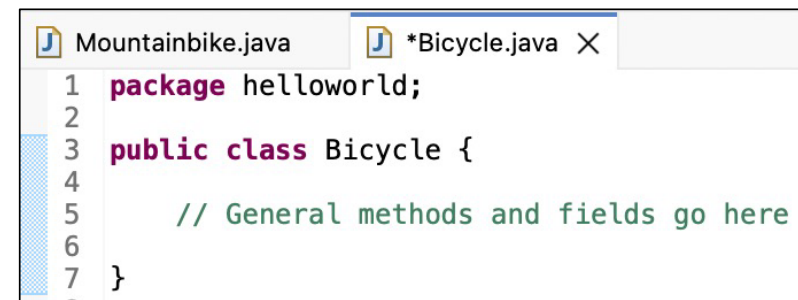


HOW DOES IT WORK?

- In Java, it is simple - just ***extend*** the new class with the parent one
 - Voila, I don't need to write all the Bicycle code for the Mountainbike. It is already there.
- Then all attributes and methods from parent becomes a part of the new class
 - With all there is some limitations, defined by scope (privates are not inherited)
- There is no boundary to how many time we can extend
 - At point it becomes a design problem



```
Mountainbike.java X Bicycle.java
1 package helloworld;
2
3 public class Mountainbike extends Bicycle {
4
5     // New specific methods and fields go here
6
7 }
```



```
Mountainbike.java *Bicycle.java X
1 package helloworld;
2
3 public class Bicycle {
4
5     // General methods and fields go here
6
7 }
```

SIMILARITIES - INHERITANCE?

- Name: Waste pump
- Attributes: rotational speed, lifting height, power consumption
- Operations: reset(), setSpeed(rotPerMin), getSpeed(), start(), stop()
- Hidden information: number of running hours, max flow, max current, ...



- Name: Drone
- Attributes: verticalSpeed, horizontalSpeed, powerConsumption
- Operations: flyTo(), setVertSpeed(), start(), stop()
- Hidden info: battery power, number of running hours, ...



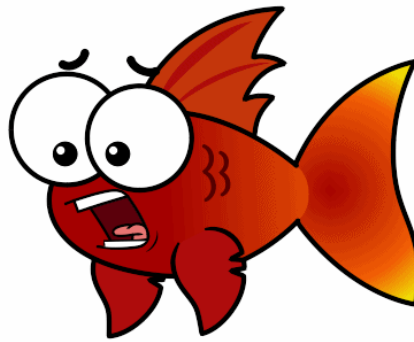
Both a pump and a drone are **controllable electro-mechanical devices** with:

- state (speed, consumption, internal counters)
- operations (start/stop/control)
- hidden internal monitoring

This makes them good candidates for **inheritance from a shared superclass**.

BREAK – PAUSE(FISK)

12



ATTRIBUTE AND METHOD VISIBILITY

- For single classes we may protect or not protect attributes and methods by keywords
 - Public
 - Private
- But for inherited classes, we might want to use parent methods and access parent attributes, but keep the to ourselves
 - Protected

Access level	Accessible to		
	Same class	Subclass	Not a subclass
Public	Yes	Yes	Yes
Protected	Yes	Yes	No
Private	Yes	No	No

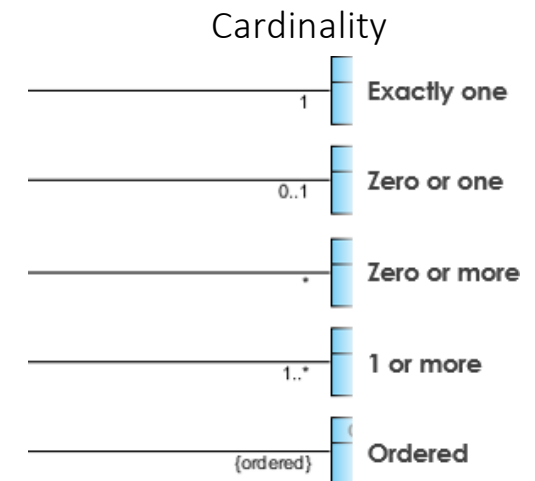
AGENDA

- Class interactions and relations
 - Inheritance
 - Associations
 - Aggregations
 - Composition
- How to complete classdiagrams
- Example
- The important stuff
- Assignments

CLASS DIAGRAMS - ASSOCIATION

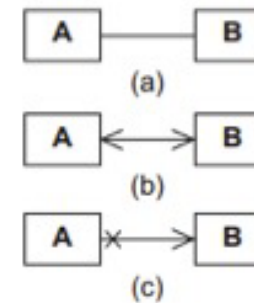
A knows B relation

- A person might have (know) a bicycle or two, but not necessarily
 - Person *knows* Bicycle
 - Bicycle does not know Person
 - Person can invoke methods in Bicycle
- Cardinality tell us how objects are connected
 - A person has exactly one Passport
- Arrows may also be used and not just lines and here we must take note of the direction to fully understand the nature of the relation

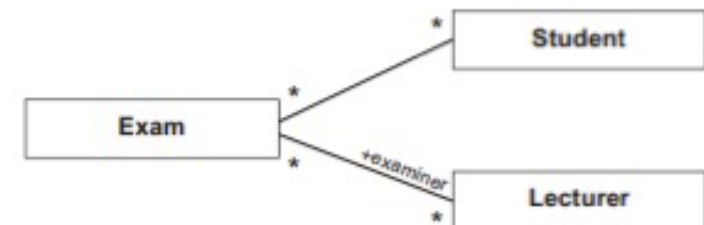


CLASS DIAGRAMS - ASSOCIATION

- For more specific associations, we can:
 - Add (open) arrows and crosses to
 - make directionality explicit or binary (one-way)
 - Annotate so that the intent of the association is communicated
 - State the exact property name and visibility



Order of the ends and reading: Car - was designed in - Year



HOW DOES AN ASSOCIATION WORK?

- Association is the most generic type of composition of objects – all it requires is a class to know another class

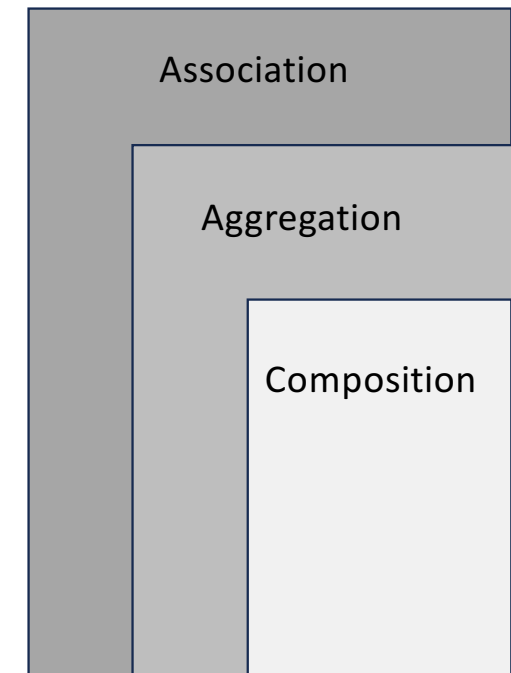
```

30 private boolean translateLocation = false;
31 private boolean fullInfo = false;
32 private boolean toRepeat = true;
33 private Vector<CIMNode> queryResultCIMNodes = null; //To hold the results in..
34 private CIMTree queryResultCIMTree = null; //To hold a useful cim tree
35 private CIMReader cimFileReader = null;
36 private Stack<CIMObject> cimObjectHistory = new Stack<CIMObject>(); //To hold the history
37 private CIMObject cimObj = null;
38

```

No association – simple datatypes

Associations to Vector, CIMTree, ...



- Association < Aggregation < Composition
- Association: “These two interact somehow.”

AGENDA

18

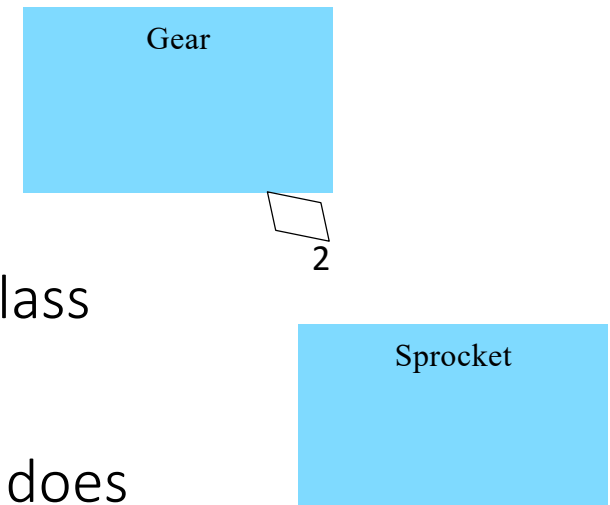
- Class interactions and relations
 - Inheritance
 - Associations
 - Aggregations
 - Composition
- How to complete class diagrams
- Example
- The important stuff
- Assignments

CLASS DIAGRAMS - AGGREGATION

19

A part of B relation

- One of several specific forms of association
- This shows that a class has objects of another class, e.g. a gear has two sprockets
 - Sprocket is *part of* Gear
- In other words, a form of ownership
- Useful to determine what class relates to what class
- Gear can invoke actions in Sprocket
- Aggregation: “The “whole” groups the parts but does not own their lifetime.”



CLASS DIAGRAMS - AGGREGATION

- Aggregation is different from, and a stronger form of "association" than, association
- The difference is that this type of association does not involve owning and the lifecycle of objects are not tied to each other
 - Garbage collection does not necessarily happen, and objects may hang around occupying memory
- Examples could be:
 - The bicycle and its wheels can be separated
 - The Robator can run without the Drone
- Both cases seem a bit useless – but that is why there is an aggregation in the first place
- In both cases, the lack of one does not destroy or prevent the other to exist

2

HOW DOES AGGREGATION WORK?

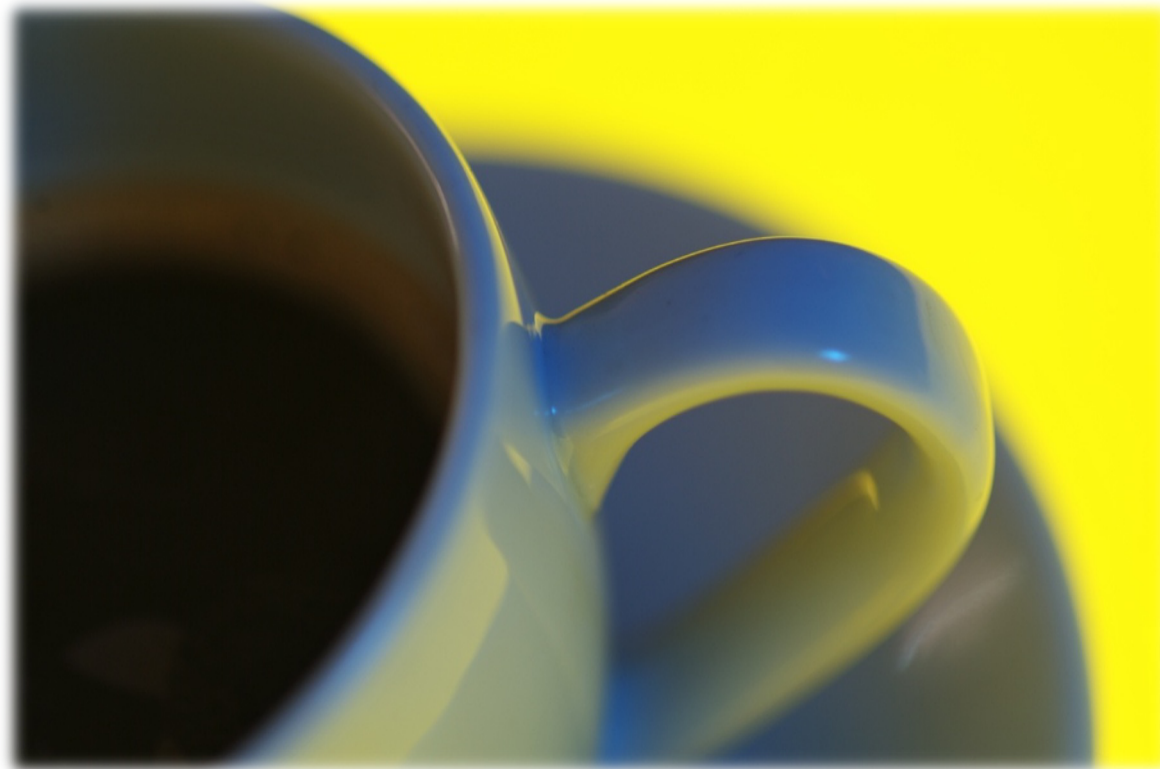
- Another example could be a bank that keeps track of its employees
- The Bank knows (many) Employee(s)
 - Despite being created in another class
- Employees are a part of the bank
 - We can have a bank with no employees
 - We can also have employees without a bank
 - Is it realistic and does it make sense – no, but it may still be feasible
- If we remove employees, we can just remove them in the main code
 - Would be a bad coding style though, because we allow external manipulation with an internal attribute (employees) in another class (the Bank)

```
public static void main(String[] args)
{
    // Creating objects of bank and Employee class
    Bank bank = new Bank("CITI");
    Employee emp = new Employee("Ridhi");

    bank.addEmployee (emp);
    ....
}
```

```
class Bank {
    private String name;
    private Set<Employee> employees;
    public Bank(String name) {
        this.name = name;
    }
    public void addEmployee(Employee emp){
        employees.add(emp);
    }
    ...
}
```

"BIKER BREAK!"



AGENDA

- Class interactions and relations
 - Inheritance
 - Associations
 - Aggregations
 - Composition
- How to complete class diagrams
- Example
- The important stuff
- Assignments

CLASS DIAGRAM - COMPOSITION

24

- A special and stronger form of aggregation with added dependency
 - If A is destroyed, then B is also destroyed
 - B cannot exist without A, since B is a part of A
 - The life-cycle of B is controlled by A
- If I destroy the Gear, then the Sprocket is also destroyed
 - Well, we work with models ... in real life we could probably reuse the sprocket
- Composition: “The parts cannot live without the “whole”.”

A has a B relation

Gear

2

Sprocket

CLASS DIAGRAM - COMPOSITION

- Compared to the association, then the composition is a very strong association where the objects lifecycle is tied together
- If we destroy the owner, we also destroy the members
 - Garbage collection happens and objects are cleared from memory
- As examples:
 - A building has rooms – If we tear down a building, its rooms go with it.
 - A university has faculties – if AAU were to close, so would its faculties!
- Semantically the difference may be subtle compared to aggregation, but technically it is more clear

CLASS DIAGRAM - COMPOSITION

- With a little change we also change the relation
 - A Bank has employee(s)
 - Now an employee cannot exist without a bank
 - And if we destroy the bank, then we also destroy the employees
- If we want to remove employees, we would need to add some methods to the Bank class to handle this operation

```
public static void main(String[] args)
{

    // Creating objects of bank and Employee class
    Bank bank = new Bank("CITI");
    bank.addEmployee ("Ridhi");
    ....
}
```

Aggregation:
Caller owns Employee
but the Bank merely
references it

```
class Bank {
    private String name;
    private Set<Employee> employees;
    public Bank(String name) {
        this.name = name;
    }
    public void addEmployee(Employee emp){
        employees.add(emp);
    }
    ...
}
```

```
class Bank {
    private String name;
    private Set<Employee> employees;
    public Bank(String name) {
        this.name = name;
    }
    public void addEmployee(String name){
        Employee emp = new Employee(name);
        employees.add(emp);
    }
    ...
}
```

Composition: Bank owns Employee

“REAL LIFE” EXAMPLE

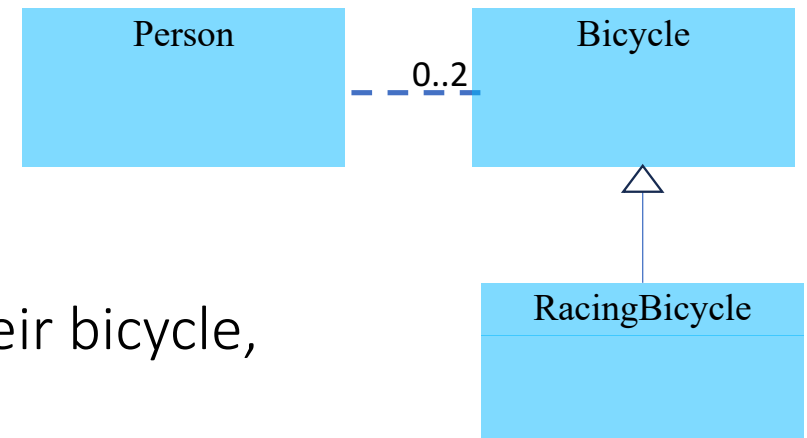
- Smart Home Management System - A single app controls devices across the house
- Why associations matter here
 - Different smart-home devices have different relationships to the system
- Composition (strong ownership)
 - A SmartLightController owns its LightBulbs
If the controller is removed, the bulbs are useless in this setup
- Aggregation (weak whole-part)
 - A Room aggregates SmartDevices
Devices can be moved from one room to another and still function
- Association (just a link)
 - A SmartDevice is associated with a UserAccount so the user can operate the device
Removing a user does NOT remove the device, and removing the device does NOT delete the user
- Why it matters - if modelled incorrectly:
 - Deleting a room might accidentally delete all devices (wrong composition)
 - Devices might not be reusable across rooms (poor aggregation model)
 - User/device associations might accidentally imply ownership (wrong lifecycle)
 - This leads to bad design decisions, bugs, or wrong business logic

“REAL LIFE” EXAMPLE

- Scenario 2: Learning Management System (LMS)
 - A system like Moodle or Canvas
- Composition
 - A Course is composed of Modules or Assignments
If the course is deleted, its assignments must go too
- Aggregation
 - A Teacher aggregates Courses they teach
Courses still exist if a teacher leaves
- Association
 - A Student is associated with Courses they are enrolled in
Many-to-many with independent lifecycles

DEPENDENCY

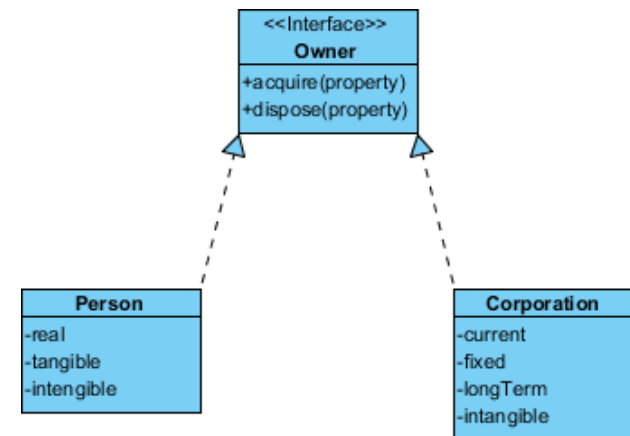
- A special type of association, where either
 - changes in A also impacts B
 - A relies on B for some functionality
- If a person age (and they do), then the bicycle would at some point also need to change
- Alternatively, if a person wants to Ride() their bicycle, they need to have one in the first place
- Any changes in Bicycle, has immediate effect on RacingBicycle
 - Without doing anything due to inheritance!!
- Any changes in RacingBicycle do NOT impact Bicycle
 - It only works one way



REALIZATION

- A relation that points out who has the blueprint and who has the code realization of the blueprints (**interface***)
- Methods may be the same for acquiring and disposing properties for Persons and Corporation
 - But the implementation may be very different
- Useful if we wish to interact with objects the same way, even though implementations are very different
 - E.g. share Drive() interface across bicycle, car, train, etc.

A implements B relation



<https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/>

*) future topic

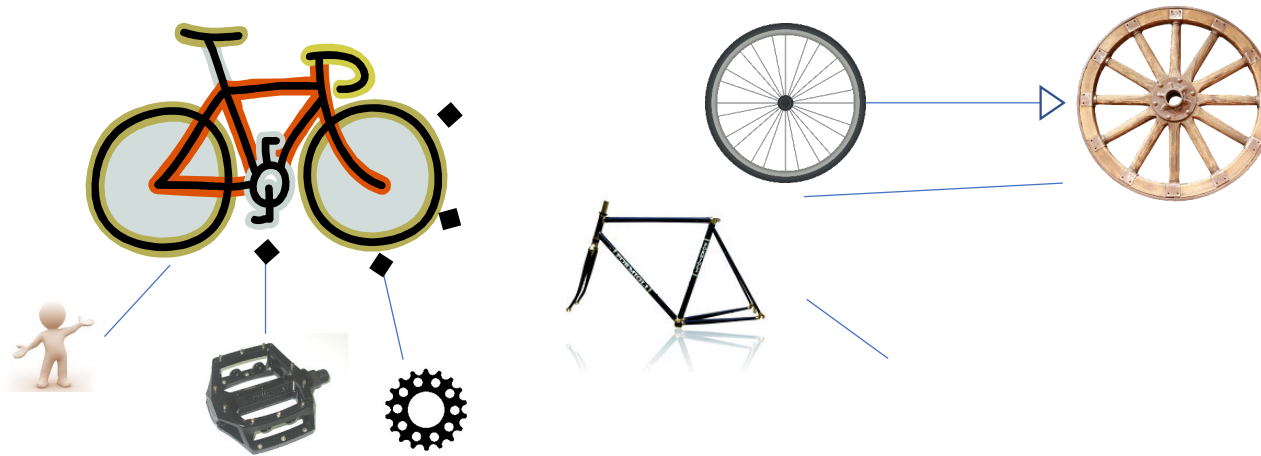
AGENDA

31

- Class interactions and relations
 - Inheritance
 - Associations
 - Aggregations
 - Composition
- How to complete class diagrams
- Example
- The important stuff
- Assignments

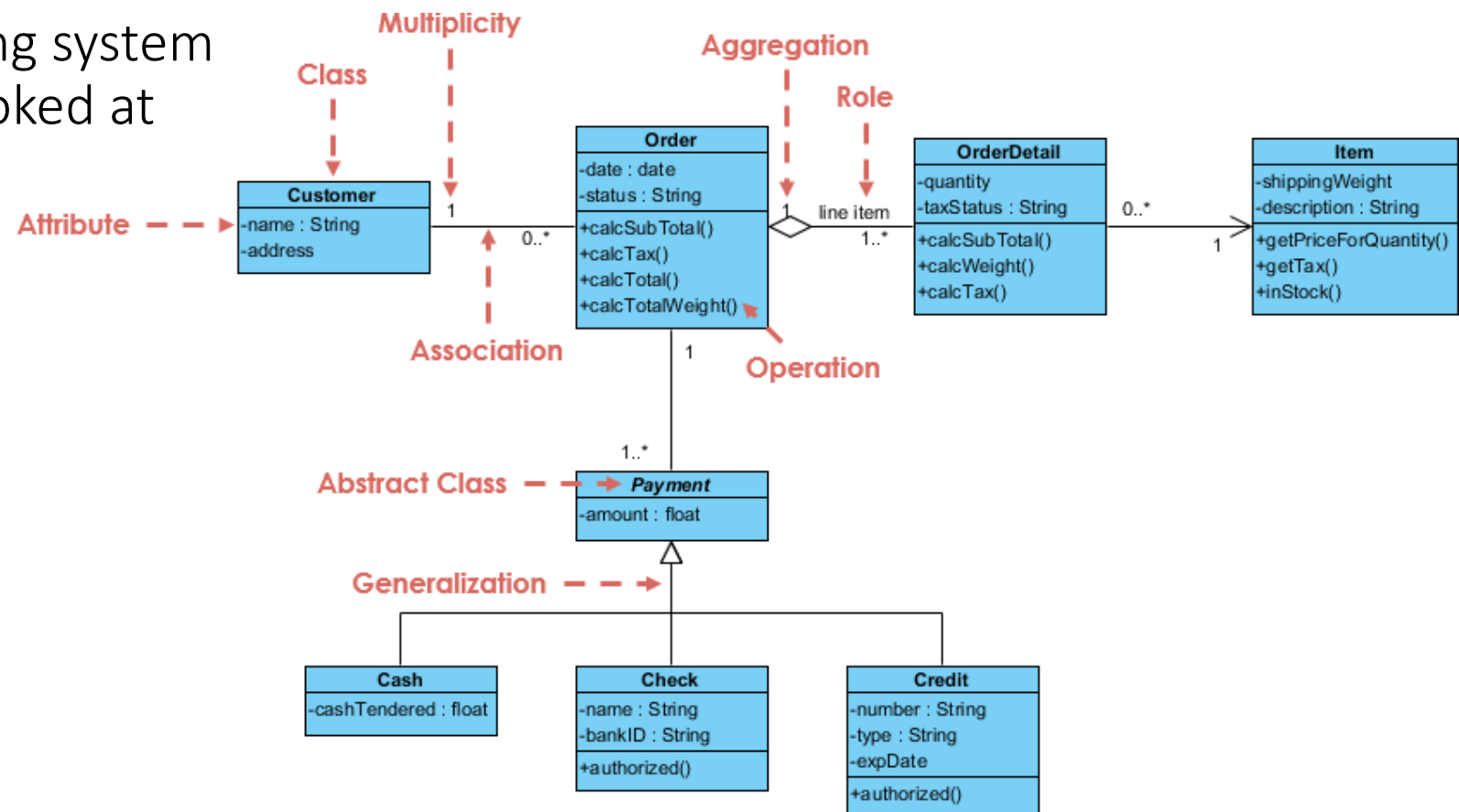
PUTTING IT TOGETHER – CLASS DIAGRAMS

- Problem: we have a ton of classes and no it all seems like a big mess
- Class diagrams are graphical views to capture our OO design
 - How does object relate to other objects

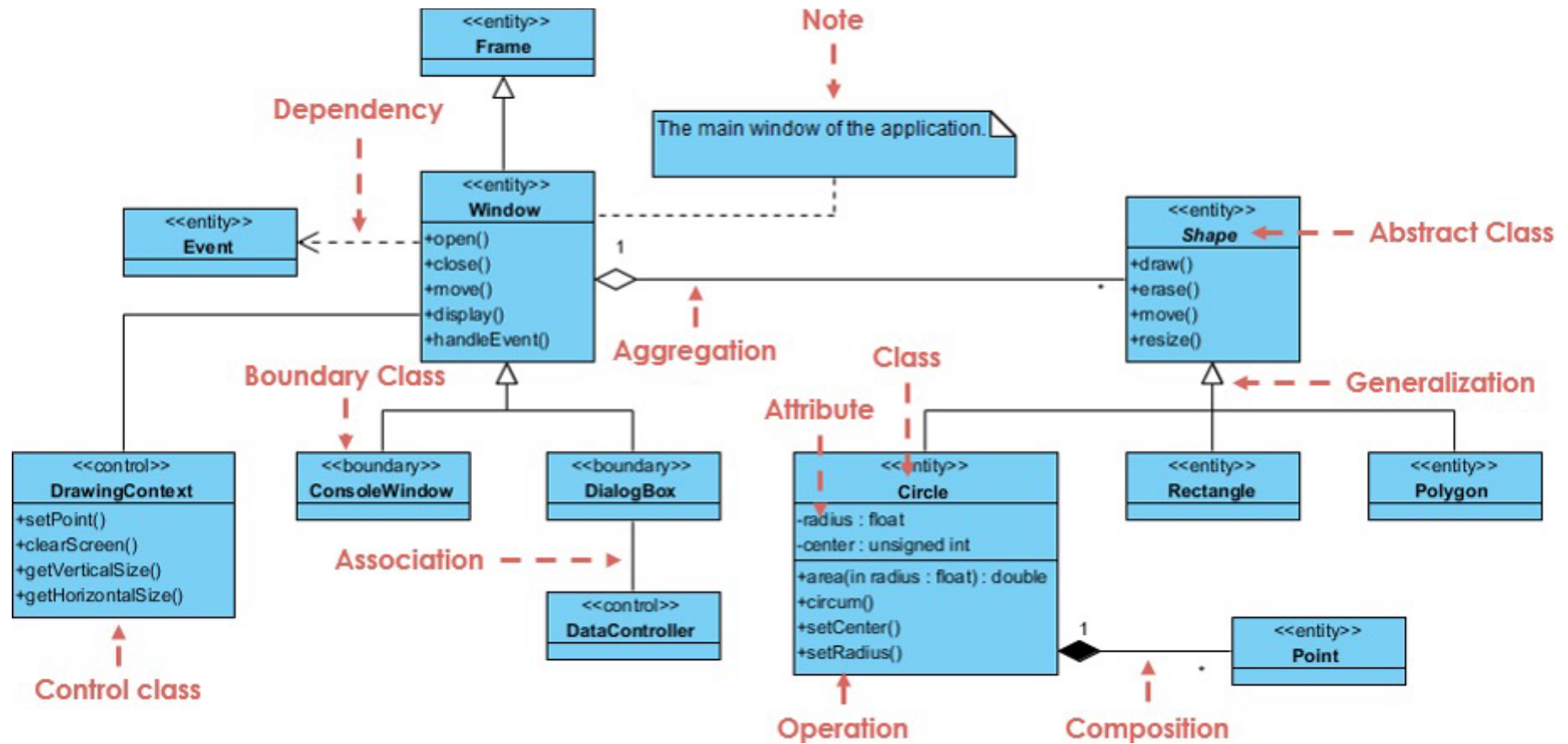


CLASS DIAGRAM EXAMPLE – ORDER SYSTEM

- Here we can return to the ordering system that we looked at last time

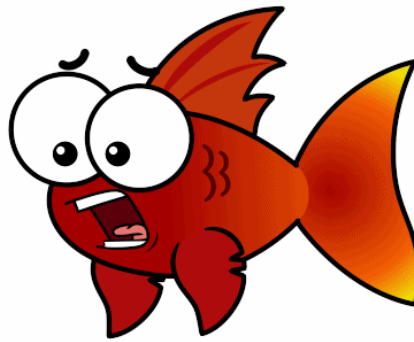


CLASS DIAGRAM EXAMPLE – GUI



BREAK – PAUSE(FISK)

36



DESIGN PROCESS – TOWARDS A CLASS DIAGRAM

- First, this is a design process and most likely you need to iterate several times to get to a satisfying result
- What is a good class diagram? By intuition (and non-measurable!!)

It is neither too small nor too big
It is descriptive and not too complex
It relates to our code and is consistent
It relates to the real world and is consistent

- But how do we achieve this???



MODELING THE WORLD

- Defining a hierarchy:
 - For each class ask
 - Is there a generic version of this class?
Yes: then generalize it, No: do nothing. End algorithm if generalization is just an object of some sort (this is the root anyway).
 - Does a method require specialized or same type of implementation?
If specialized, then consider an interface instead of a parent class.
- Construct the tree – e.g. on a black board (faster to work this way)
Some trees may stand alone, and some classes are on their own – that is just fine
 - For each class ask
 - Is there a relation to other classes?
Yes, then decide which type and draw this.
No, then leave alone

Must be emphasized! It is tempting to jump straight into a diagram program, but you risk spending more time “fighting the software” then sketching your diagrams. Start simple, with generic illustrations and only once this is done, then consider making it nice.

MODELING THE WORLD

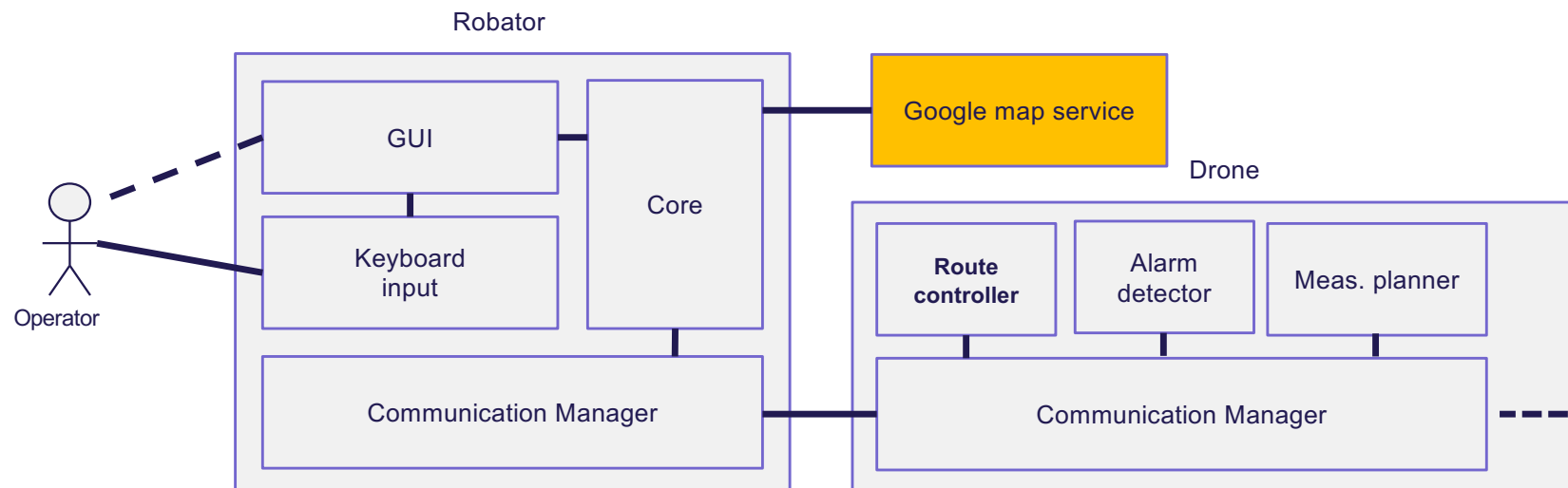
- Give it a go with coding empty classes
 - Does it make sense? If no, then iterate
- Iterate when needed
 - But be aware, the longer you wait, the harder it get to change things
 - Refactoring is a friend!!! But use it with care – with great power comes great responsibility!!

AGENDA

- Class interactions and relations
 - Inheritance
 - Associations
 - Aggregations
 - Composition
- How to complete class diagrams
- Example
- The important stuff
- Assignments

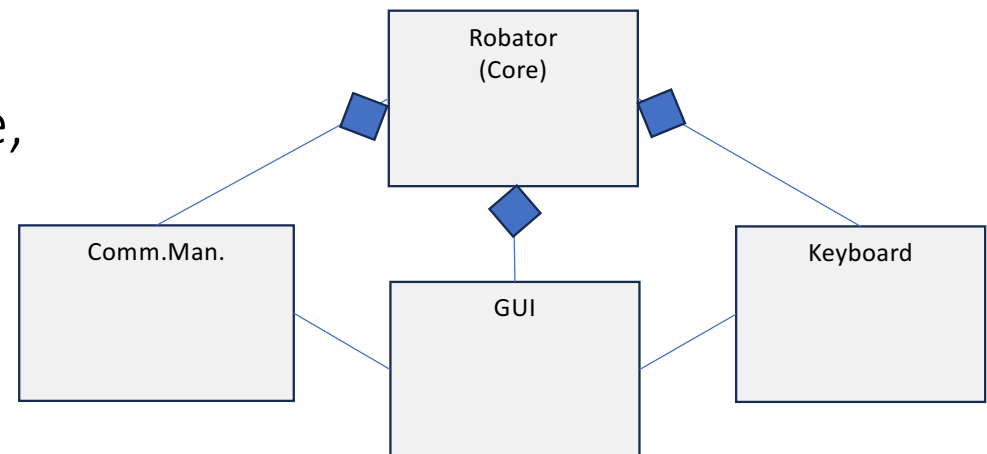
ROBATOR EXAMPLE

- Let us revisit the Robator example from previously
- We managed to work ourselves through use-case diagrams and what have you, to end up with this modular overview
- Now say that we want to modelled each module as a class



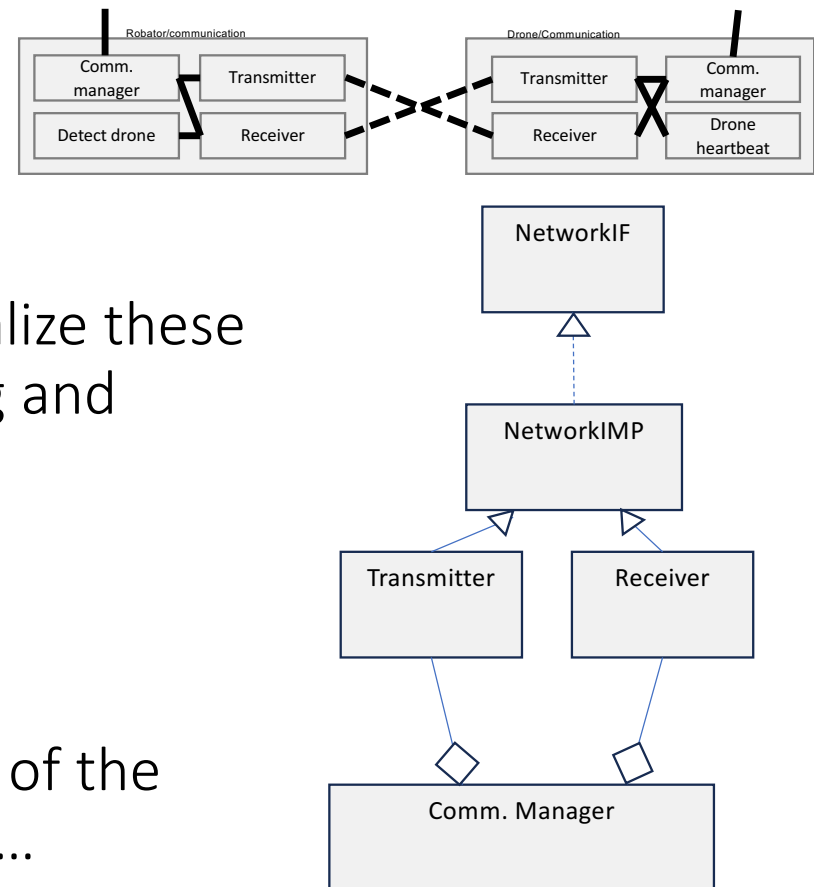
ROBATOR EXAMPLE

- For all internal modules it seems appropriate to have some sort of aggregation to the core part of Robator
- A composition seems appropriate, since the closure of Robator, should also lead to closure of the other modules
- The GUI may be aware of the Comm. Manager and Keyboard, as it will need to show stuff based on what goes on there



ROBATOR EXAMPLE

- The transmitter is responsible for sending packet, and the receiver is responsible for receiving data. They do share some network related parts
- It therefore seems reasonable to generalize these two and have a specialization of sending and receiving data. We can
 - Define an interface
 - Create a parent class
 - Or both
- Both Transmitter and Receiver are parts of the Comm. Manager ... and so we continue ...



AGENDA

- Class interactions and relations
 - Inheritance
 - Associations
 - Aggregations
 - Composition
- How to complete class diagrams
- Example
- The important stuff
- Assignments

THE IMPORTANT STUFF

- Class diagrams describe structure not behaviour
 - Classes, attributes, methods, and relationships
- Multiplicity matters and cardinality defines “how many”?
 - It defines how many objects participate in a relationship
- Relationship types differ, in strength and in ownership
 - Association < Aggregation < Composition
- Use inheritance only for shared behaviour, not just shared method names
 - If each subclass implements behaviour differently, prefer an interface or composition over inheritance
- Good class design mirrors reality and guides clean, maintainable, real-world implementations

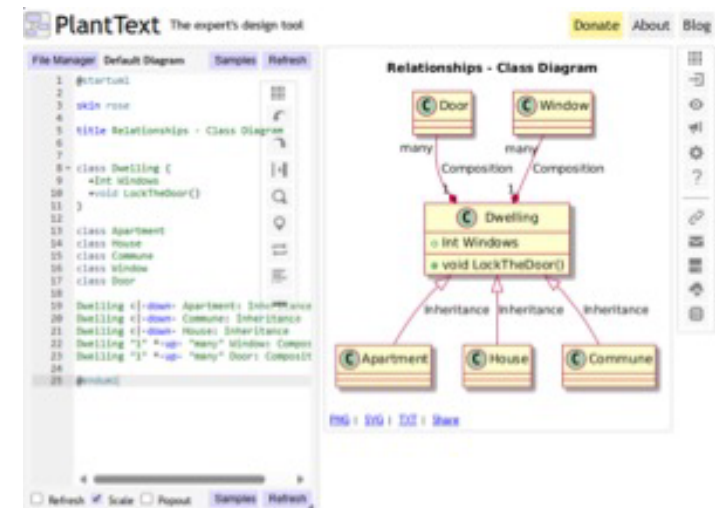
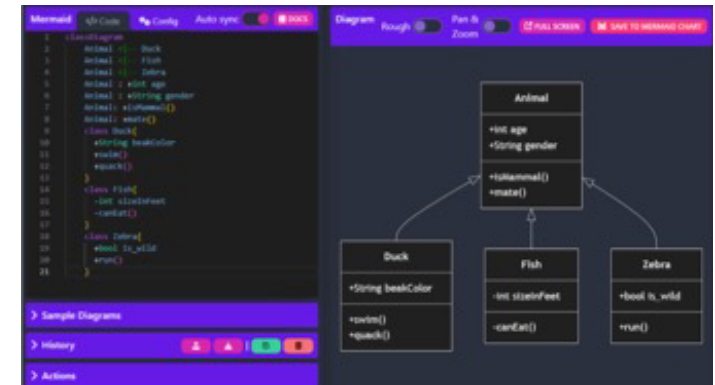
- <https://www.drawio.com/>



BONUS

47

- Other options are some of the (many) textual model programs where you “code” a diagram
- It requires learning a separate syntax
- <https://mermaid.js.org/> is a notable mention
- Surface-level syntax is not that far from what we’re currently doing, and it is directly supported in GitHub markdown
- A similarly popular alternative is PlantUML, which is a fairly accessible online implementation
 - <https://www.planttext.com/>



AGENDA

- Class interactions and relations
 - Inheritance
 - Associations
 - Aggregations
 - Composition
- How to complete class diagrams
- Example
- The important stuff
- Assignments



ASSIGNMENTS

1. Draw a class diagram of the bicycle from last time
2. Think of a larger group of bicycle types. Create a class hierarchy of the bicycles – both subclasses and super classes. Create methods and attributes for super and subclasses that are specific to that level. Top super class is always Object
3. Implement at least three subclasses and one super class to the bicycle class
4. Instantiate several types of the bicycle class and demonstrate by code execution that you are using three different bicycle types. Use methods from all levels of the defined hierarchy
5. Discuss the difference between the keywords this and super. Does it make sense to have a keyword sub?